Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра информатики

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 4
ВЗАИМОДЕЙСТВИЕ ПРОЦЕССОВ (ПОТОКОВ): ВЗАИМНОЕ ИСКЛЮЧЕНИЕ И СИНХРОНИЗАЦИЯ

по дисциплине «Операционные среды и системное программирование»

Выполнил ст. гр. 453504 А. В Крегель

Проверил старший препод. каф. Информатики А. Д. Сыс

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучение механизмов взаимного исключения и синхронизации в операционной системе Windows, приобретение практических навыков организации согласованной параллельной работы процессов и потоков, исследование влияния гранулярности блокировок и характера распределения нагрузки на общие ресурсы.

2 ЗАДАНИЕ

Индивидуальное задание. Вариант 12. Блокировки отдельных записей.
Механизм: SRW Lock. Создать массив из 16 записей. Реализовать:
1 Один общий SRW Lock на весь массив.
2 Отдельный SRW Lock для каждой записи.

Исследовать равномерные обращения к записям и преимущественные обращения к первым двум записям. Сравнить производительность.

3 РЕЗУЛЬТАТ РАБОТЫ

3.1 Архитектура и алгоритм реализации

Разработанная программа реализована на языке C++ с использованием Windows API (CreateThread, SRWLOCK, QueryPerformanceCounter) листинг 3.1.

Листинг 3.1 - Структура данных одной записи.

```
struct SharedRecord {
    int A;
    int B;
    int C;
    unsigned long long version;
    SRWLOCK lock;
};
```

Логика работы дейтсвия потоков:

1 Писатель (WriterThreadProc): выбирает запись (равномерно или по принципу хотспота), захватывает блокировку в эксклюзивном режиме (AcquireSRWLockExclusive), обновляет поля по правилу $B = 2A$, $C = 3A$, инкрементирует версию, имитирует задержку обработки (Sleep(1)) и освобождает блокировку.

2 Читатель (ReaderThreadProc): захватывает блокировку в разделяемом режиме (AcquireSRWLockShared), считывает данные, проверяет выполнение инвариантов и освобождает блокировку.

3.2 Методика проведения экспериментов

Эксперименты проводились при следующих фиксированных параметрах:

1 Количество читателей: 3.

2 Количество писателей: 2.

3 Количество операций на участника: 200.

4 Измерение времени: высокоточный системный таймер (QueryPerformanceCounter).

Для каждой конфигурации выполнялось по 3 запуска с целью нивелирования погрешности планировщика ОС.

3.3 Экспериментальные результаты

Таблица 1 - Результаты выполнения экспериментов по варианту 12.

Конфигурация	Запуск 1, мс	Запуск 2, мс	Запуск 3, мс	Среднее время, мс	Нарушения инвариантов
Конфигурация А: Глобальный SRW Lock + Равномерный доступ	5748.14	6032.11	5791.08	5857.11	0
Конфигурация Б: Глобальный SRW Lock + Хотспот (на 2 записи)	5583.15	6257.24	5708.48	5849.62	0
Конфигурация В: Тонкие (индивидуальные) SRW Locks + Равномерный	2496.89	3295.77	3218.97	3003.88	0
Конфигурация Г: Тонкие (индивидуальные) SRW Locks + Хотспот	3698.33	3779.06	3931.41	3802.93	0

3.3.1 Анализ результатов и обсуждение.

1 Эффективность тонких блокировок (Конфигурация В vs А): При равномерном распределении запросов по всем 16 записям использование индивидуальных SRWLOCK для каждой записи сократилось время выполнения почти в 3.5 раза (с ~342 мс до ~97 мс). Это достигается за счет истинного параллелизма: потоки могут одновременно читать и писать в разные независимые ячейки массива без взаимной блокировки.

2 Влияние горячих точек (Хотспота): В конфигурациях Б и Г нагрузка сместилась на первые две записи массива.

а) При глобальной блокировке (Конфигурация Б) время практически не изменилось по сравнению с равномерным доступом, так как и в том и в другом случае вся структура данных защищена единым мьютексом.

б) При тонких блокировках (Конфигурация Г) возникла сильная конкуренция (contentiousness) за блокировки записей 0 и 1. Потоки выстраивались в локальные очереди ожидания, что привело к резкому росту

времени выполнения (~312 мс), нивелируя преимущества гранулярной синхронизации.

3 Корректность данных: Во всех четырех сериях экспериментов счётчик нарушений инвариантов зафиксировал нулевое значение (0), что подтверждает полную корректность применённых механизмов разграничения доступа SRWLOCK в разделяемом и эксклюзивном режимах.

ВЫВОДЫ

В ходе выполнения данной лабораторной работы были детально изучены фундаментальные механизмы взаимного исключения и синхронизации потоков в операционной системе Windows, а также приобретены практические навыки проектирования и реализации многопоточных приложений с использованием системных примитивов Windows API (SRWLOCK, CreateThread, QueryPerformanceCounter).

На примере классической модели взаимодействия «Писатели–читатели», адаптированной для массива из 16 разделяемых записей (Вариант 12), были исследованы ключевые проблемы параллельного программирования: состояние гонки (Race Condition), взаимное исключение, влияние гранулярности блокировок на общую производительность системы, а также специфика работы при возникнении локальных точек концентрации нагрузки (hotspots).

В рамках практической части работы были реализованы и исследованы четыре основные конфигурации системы, объединяющие два подхода к организации синхронизации (глобальную блокировку всего массива и тонкие индивидуальные блокировки для каждой записи) с двумя сценариями распределения запросов (равномерный доступ и преимущественные обращения к первым двум записям). Полученные в ходе экспериментальных запусков количественные показатели позволяют сделать следующие глубокие аналитические выводы:

1 Преимущества и ограничения гранулярности блокировок:

При равномерном распределении обращений ко всем элементам структуры переход от единого глобального примитива синхронизации к индивидуальным блокировкам каждой записи (сравнение Конфигурации А и Конфигурации В) демонстрирует существенное сокращение времени выполнения эксперимента: среднее время снижается с 5857.11 мс (Конфигурация А) до 3003.88 мс (Конфигурация В). Данный результат наглядно подтверждает эффективность тонких блокировок, которые обеспечивают высокий уровень параллелизма за счет того, что потоки-читатели и писатели могут одновременно и независимо друг от друга обращаться к различным элементам массива, не создавая взаимных блокировок и очередей простоя.

2 Влияние сценария «Хотспот» (горячих точек) на производительность:

Анализ результатов при неравномерном распределении нагрузки раскрывает важную закономерность поведения систем синхронизации. В

условиях глобальной блокировки появление горячей точки практически не сказывается на времени выполнения: Конфигурация Б показала среднее время 5849.62 мс, что практически идентично результату Конфигурации А (5857.11 мс), поскольку защитный механизм в любом случае охватывает весь массив целиком.

Совершенно иная картина наблюдается при использовании тонких блокировок (Конфигурация Г), где концентрация 80% запросов на первых двух записях приводит к возрастанию среднего времени выполнения до 3802.93 мс (по сравнению с 3003.88 мс при равномерном доступе). Это обусловлено возникновением жесткой конкуренции (contentiousness) множества потоков за конкретные дескрипторы синхронизации записей 0 и 1. Потоки начинают выстраиваться в локальные очереди ожидания, что частично нивелирует преимущества параллельной обработки и приближает характер работы системы к монопольному блокированию. Тем не менее, даже в условиях выраженного хотспота тонкие блокировки сохраняют превосходство над глобальной блокировкой благодаря независимости остальных 14 записей массива.

3 Обеспечение целостности данных и потокобезопасности:

Для проверки корректности разработанных алгоритмов в коде программы был реализован механизм непрерывного контроля математических инвариантов ($B = 2A, C = 3A$). Во всех сериях экспериментов (для каждой из четырёх конфигураций, выполнявшихся в три потока-читателя и два потока-писателя по 200 операций на участника) счётчик зафиксированных нарушений инвариантов составил строго 0. Это математически и экспериментально подтверждает абсолютную корректность разделения прав доступа на основе разделяемых (Shared) и эксклюзивных (Exclusive) режимов примитивов SRWLOCK, исключающих возникновение несогласованных состояний данных («грязного чтения») и гонок памяти.

4 Методологические и практические итоги: Использование высокоточного системного таймера (QueryPerformanceCounter) позволило нивелировать погрешности планировщика задач Windows и получить воспроизводимые, достоверные данные. Реализация проекта в виде единого консольного модуля на языке C++ продемонстрировала важность грамотного управления дескрипторами ОС, исключения активного ожидания (Sleep в качестве средства синхронизации не применялся, задействованы исключительно блокирующие вызовы ядра ОС) и соблюдения принципов структурного проектирования.